reactive stream, it will wrap it in one nevertheless and return a `Mono<Mono<Order>>` object. On the other hand, the `flatMap()` operator expects the target type to be a reactive stream, so it knows how to handle the publisher produced by `OrderRepository` and returns a `Mono<Order>` object correctly.

In the next section, you'll complete the basic implementation of Order Service by exposing an API to fetch and submit orders.

8.2.4 Exposing a REST API with Spring WebFlux

There are two options for defining RESTful endpoints in a Spring WebFlux application: `@RestController` classes or functional beans (router functions). For the Order Service application, we'll use the first option. Unlike what we did in chapter 3, the method handlers will return reactive objects.

For the `GET` endpoint, we can use the `Order` domain entity we defined earlier and return a `Flux<Order>` object. When submitting an order, the user must provide the ISBN of the desired book and the number of copies they would like to purchase. We can model that information in an `OrderRequest` record that will act as a data transfer object (DTO). It's also good practice to validate the input, as you learned in chapter 3.

Create a new `com.polarbookshop.orderservice.order.web` package, and define an `OrderRequest` record to hold the submitted order information.

Listing 8.14 The `OrderRequest` DTO class with validation constraints

```
package com.polarbookshop.orderservice.order.web;

import javax.validation.constraints.*;
```